2026-10-17 · 第22节 · 学习任务管理系统 · 图文与代码练习

点击“创建”后，任务怎样进入数据库？把 Vue 新建任务接上后端

上一节理解了页面打开时，Vue 通过 `GET /api/tasks` 读取数据库。今天反过来追踪用户填写弹窗并点击创建后发生的事情：表单数据变成 JSON，请求带着 JWT 到达后端，后端验证并插入 MySQL，成功后前端重新读取列表。

> **今天的目标：**看懂并能跟写“表单 → POST → Controller → Service → Mapper/MyBatis → MySQL → 刷新列表”这一条链。浏览器里能创建任务只是结果，真正要会解释数据经过了哪些层。

先看第 1 ~ 3 节地图，再看第 4 节真实代码。第 5 节先在草稿中跟写，第 6 节做一次安全验证。今天不进入编辑、删除和统计的新功能。

1. 先看整体：一次新建任务的完整旅程

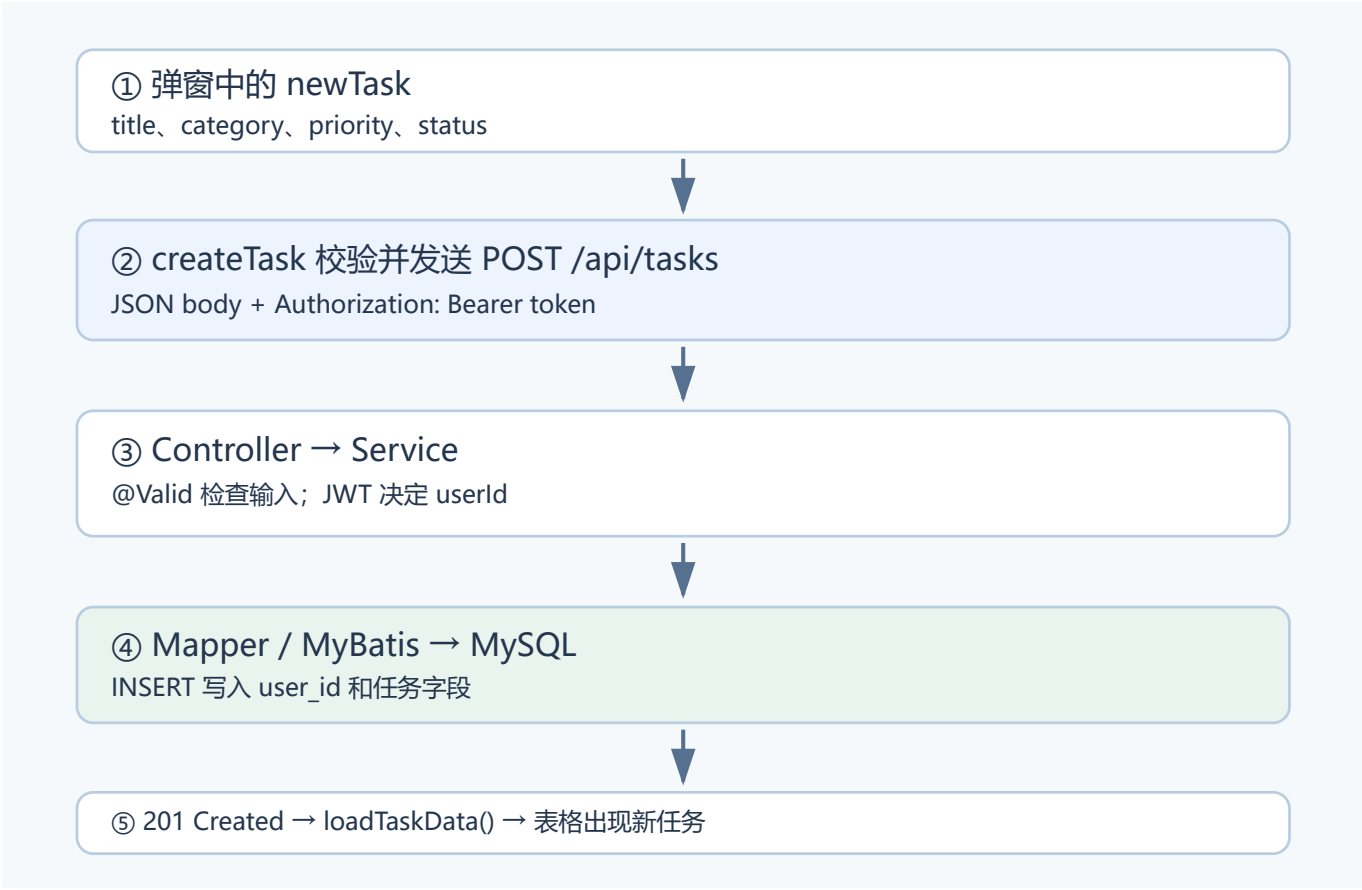


图 1：前端提交的是任务内容，后端从 token 取得归属用户；客户端不提交也不能决定 user_id。

可以把它和昨天的查询对照：查询是 `GET` 把数据库数据拿到页面；今天是 `POST` 把页面填写的数据送到数据库。

2. 弹窗里的数据到底是什么?

```
const newTask = ref({
  title: "",
  category: "Web",
  priority: "普通",
  status: "未开始",
});
```

这是 Vue 的表单状态。输入框用 `v-model="newTask.title"` 双向绑定，所以用户输入文字时，`newTask.title` 同步变化。下拉框和单选框同理。

字段	页面中的来源	后端用途
title	输入框	任务名称，不能空白，最多 200 字
category	分类下拉框	只能是 Web、Java、AI
priority	优先级单选框	只能是普通、高
status	新建时默认未开始	只能是未开始、进行中、已完成
user_id	页面不提供输入	后端从已验证 JWT 得到

> 页面的 `newTask` 只是尚未提交的内存对象；调用 API 成功并写入 MySQL 后，才成为持久记录。不要看到对象里没有 `user_id` 就认为后端不知道用户是谁。

3. 为什么需要前后端两次校验?

当前页面在 `createTask()` 开始处检查空标题，是为了让用户马上得到反馈、避免无意义请求：

```
if (!newTask.value.title.trim()) {
  ElMessage.warning("请输入任务名称");
  return;
}
```

但用户可以绕过页面，直接用 PowerShell 或其他程序请求接口。因此后端的 `TaskRequest` 仍有 `@NotBlank`、`@Size` 和 `@Pattern`，Controller 的 `@Valid` 会触发检查。

校验位置	作用	失败时
Vue createTask	快速提示用户，减少错误请求	不发送 POST
Spring Boot TaskRequest + @Valid	保护接口，不信任客户端	返回 400，数据库不新增
MySQL 约束与事务	保存层最后一道保护	写入失败，由后端处理错误

所以今天验证空标题时，Network 没有 POST，只证明前端拦截；它不能代替后端 400 的接口验证。

4. 沿真实代码追踪“创建”分支

4.1 createTask：区分新建和编辑

```
async function createTask() {
  if (taskBusy.value) return;
  if (!newTask.value.title.trim()) {
    ElMessage.warning("请输入任务名称");
    return;
  }
  taskBusy.value = true;
  const editing = editingTaskId.value !== null;
  try {
    await apiRequest(
      editing ? `/api/tasks/${editingTaskId.value}` : "/api/tasks",
      {
        method: editing ? "PUT" : "POST",
        body: JSON.stringify({
          ...newTask.value,
          title: newTask.value.title.trim(),
        }),
      },
    );
    dialogVisible.value = false;
    await loadTaskData();
    ElMessage.success(editing ? "任务修改成功" : "任务创建成功");
  } catch (error) {
    ElMessage.error(error.message);
  } finally {
    taskBusy.value = false;
  }
}
```

今天只看 `editingTaskId.value === null` 的新建分支：地址是 `/api/tasks`，方法是 `POST`。请求成功后先关闭弹窗，再重新加载任务和统计；最后才显示成功提示。

4.2 apiRequest：加上 token 和 JSON 类型

```
response = await fetch(path, {
  ...options,
  headers: {
    ...(options.body ? { "Content-Type": "application/json" } : {}),
    Authorization: `Bearer ${token.value}`,
    ...(options.headers || {}),
```

```
  },
});
```

这里没有把 `user_id` 放进 body。token 由后端验证，决定当前用户。

4.3 Controller 和 Service

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public TaskResponse create(
    @Valid @RequestBody TaskRequest request,
    @RequestHeader(value = "Authorization", required = false)
    String authorization) {
  return service.create(request, authorization);
}
```

```
public TaskResponse create(TaskRequest request, String authorization) {
    Long userId = currentUserId(authorization);
    Task task = new Task();
    task.setUserId(userId);
    task.setTitle(request.title().trim());
    task.setCategory(request.category().trim());
    task.setPriority(request.priority().trim());
    task.setStatus(request.status().trim());
    mapper.insert(task);
    return TaskResponse.from(requireOwned(task.getId(), userId));
}
```

Service 做了两件关键事：把 token 转成 userId，把请求字段复制到实体。它不接受客户端提交的归属用户。

4.4 MyBatis INSERT

```
<insert id="insert" useGeneratedKeys="true" keyProperty="id">
  INSERT INTO tasks (user_id, title, category, priority, status)
  VALUES (#{userId}, #{title}, #{category}, #{priority}, #{status})
</insert>
```

`#{...}` 取 Java 对象的属性值，MyBatis 绑定参数后执行 SQL。数据库生成的 id 会回填到 `task.id`，所以 Service 能再查询并返回完整任务。

5. 今天的跟写：先写最小创建流程

在草稿中写下面三段，不要一开始凭记忆写整个组件。每写一段，说明它解决什么问题。

A. 前端请求对象

```
const body = {
  title: newTask.value.title.trim(),
  category: newTask.value.category,
  priority: newTask.value.priority,
  status: "未开始",
};
```

B. 发送 POST

```
await apiRequest("/api/tasks", {
  method: "POST",
  body: JSON.stringify(body),
});
```

C. 后端写入对象

```
Long userId = currentUserId(authorization);
Task task = new Task();
task.setUserId(userId);
task.setTitle(request.title().trim());
mapper.insert(task);
```

跟写时先别追求完整。能说清“body 是请求内容、token 是身份、userId 由后端决定、mapper.insert 执行 INSERT”就达到本段目标。

相近小练习

在草稿中写一个 `buildCreateBody(title)`，传入标题，返回包含 `title`、`category: "Web"`、`priority: "普通"`、`status: "未开始"` 的对象。参考答案：

▶ 打开参考答案

这只是请求数据构造练习，不要把它额外加入正式项目。正式项目已有弹窗状态和接口调用。

6. 实操验证：一条任务从页面到数据库

检查一：Network 看到 POST

打开任务页面和 Network，创建一个标题明显的测试任务，例如“10月17创建链路测试”。筛选 `tasks`，应出现：

```
Request Method: POST
Request URL: /api/tasks
Status Code: 201 Created
Request Payload: title、category、priority、status
Request Headers: Authorization: Bearer ...
```

不要把 token 截图或发给别人；只检查它存在。

检查二：Response 和页面

POST 的 Response 应返回新任务对象，页面弹窗关闭并出现“任务创建成功”，随后列表中出现任务。刷新页面后它仍在，说明后端重新从 MySQL 查询。

检查三：数据库

用 MySQL 查看自己账号的任务记录，确认标题、分类、优先级、状态和页面一致。不要直接手动修改 `user_id` 来模拟登录。

检查四：错误路径

把标题清空后点击创建：页面提示且 Network 没有 POST。若用接口提交空标题，后端应返回 400；两次检查验证的是不同层。

> 如果 POST 返回 401，先检查登录会话；返回 400，查看请求字段；返回 500，查看 Spring Boot 窗口。不要只看到网页没有新行就断定数据库写入失败。

7. 复盘与完成标准

尝试不用看讲义回答：

- 为什么新建任务请求里不包含 `user_id`？
- 为什么创建成功后还要调用 `loadTaskData()`，直接把表格前端数组加一项不行吗？
- 前端空标题拦截和后端 `@Valid` 各自防什么？
- 为什么创建接口返回 201，而删除接口返回 204？

层级	完成证据
读懂	能画出弹窗 → POST → Controller → Service → INSERT → 201 → 刷新列表
跟写	能写出 body、POST 请求和设置 userId 的三段代码
会用	能独立构造一个合法任务请求，并用 Network、页面刷新和数据库三处互相验证

> **今天的定位：**后端创建接口已经存在，今天学习的是 Vue 如何正确调用它并确认数据真正落库。编辑、删除、统计留在对应课程，不用今天全部混在一起。